

Part 1

Problem 1.1

Derive $q(x_t|x_0)$

We want to derive $q(x_t|x_0)$ given the forward process, $q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t\mathbf{I})$. We use the reparameterization trick to recursively expand this from x_t to x_0 , so we rewrite each step:

$$\begin{aligned} x_t &= \sqrt{1 - \beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon_t, \epsilon_t \sim \mathcal{N}(0, \mathbf{I}) \\ &= \sqrt{\alpha_t}x_{t-1} + \sqrt{\beta_t}\epsilon_t \\ &= \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{\alpha_t}\sqrt{\beta_{t-1}}\epsilon_{t-1} + \sqrt{\beta_t}\epsilon_t \end{aligned}$$

Repeating this up to x_0 gives us:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sum_{i=1}^t (\sqrt{\beta_t}(\prod_{j=i+1}^t \sqrt{\alpha_j})\epsilon_i)$$

Since each $\epsilon_i \sim \mathcal{N}(0, 1)$ is independent, the sum of weighted Gaussians is still Gaussian. The variance is:

$$\begin{aligned} \text{Var}(x_t|x_0) &= \sum_{i=1}^t \beta_i (\prod_{j=i+1}^t \alpha_j) \mathbf{I} \\ &= (1 - \bar{\alpha}_t) \mathbf{I} \end{aligned}$$

Giving us the final distribution:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)\mathbf{I})$$

Derive $\lim_{T \rightarrow \infty} q(x_T)$

We want to derive $\lim_{T \rightarrow \infty} q(x_T)$. First, we marginalize out x_0 to give us $q(x_T) = \int q(x_T|x_0)q(x_0)dx_0$. Since $x_0 \sim q(x)$, we note that as $T \rightarrow \infty$, we also have that $\bar{\alpha}_T = \prod_{i=1}^T \alpha_i \rightarrow 0$, assuming that β_t is bounded away from 1.

From our previous result, $q(x_T|x_0) = \mathcal{N}(x_T; \sqrt{\bar{\alpha}_T}x_0, (1-\bar{\alpha}_T)\mathbf{I})$. Since $\bar{\alpha}_T \rightarrow 0$, the mean term also goes to 0, leaving $q(x_T) \approx \mathcal{N}(x_T; 0, \mathbf{I})$. Therefore, in the limit, $\lim_{T \rightarrow \infty} q(x_T) = \mathcal{N}(0, \mathbf{I})$. This confirms that under normal schedules, the diffusion process asymptotically converges to an isotropic Gaussian.

Problem 1.2

Derive $\tilde{\beta}_t$

We start by applying Bayes' theorem to find the reverse distribution $q(x_{t-1}|x_t, x_0)$. The forward process $q(x_t|x_{t-1})$ is Gaussian, so the reverse distribution also follows a Gaussian distribution with mean $\tilde{\mu}_t(x_t, x_0)$ and variance $\tilde{\beta}_t\mathbf{I}$.

To find $\tilde{\beta}_t$, we use Bayes' rule:

$$q(x_{t-1}|x_t, x_0) \propto q(x_t|x_{t-1})q(x_{t-1}|x_0)$$

Substituting the expressions for the forward and reverse processes, we get:

$$q(x_{t-1}|x_t, x_0) \propto \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t\mathbf{I})\mathcal{N}(x_{t-1}; \sqrt{\alpha_{t-1}}x_0, (1-\alpha_{t-1})\mathbf{I})$$

From this, the variance of the reverse process $\tilde{\beta}_t$ is derived as:

$$\tilde{\beta}_t = \frac{\beta_t(1-\alpha_{t-1})}{1-\bar{\alpha}_t}$$

Derive $\tilde{\mu}_t(x_t, x_0) = A \cdot x_t + B \cdot x_0$

To find the mean $\tilde{\mu}_t(x_t, x_0)$, we need to calculate the mean of the reverse process using the same reasoning. From Bayes' theorem, the mean $\tilde{\mu}_t$ is the weighted sum of x_t and x_0 , and we obtain:

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\alpha_{t-1}}}{1-\bar{\alpha}_t}x_t + \frac{\sqrt{\beta_t(1-\alpha_{t-1})}}{1-\bar{\alpha}_t}x_0$$

This gives the coefficients:

$$A = \frac{\sqrt{\alpha_{t-1}}}{1-\bar{\alpha}_t}, \quad B = \frac{\sqrt{\beta_t(1-\alpha_{t-1})}}{1-\bar{\alpha}_t}$$

Express x_0 as a combination of x_t and Gaussian noise ϵ_t

Using the distribution derived in (1), we can express x_0 as:

$$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} x_t - \sqrt{\frac{1 - \bar{\alpha}_t}{\bar{\alpha}_t}} \epsilon_t$$

where $\epsilon_t \sim \mathcal{N}(0, \mathbf{I})$.

Rewrite $\tilde{\mu}_t(x_t, \epsilon_t)$ as a combination of x_t and ϵ_t

Substitute the expression for x_0 from (5) into $\tilde{\mu}_t(x_t, x_0)$:

$$\tilde{\mu}_t(x_t, \epsilon_t) = A \cdot x_t + B \cdot \left(\frac{1}{\sqrt{\bar{\alpha}_t}} x_t - \sqrt{\frac{1 - \bar{\alpha}_t}{\bar{\alpha}_t}} \epsilon_t \right)$$

This simplifies to:

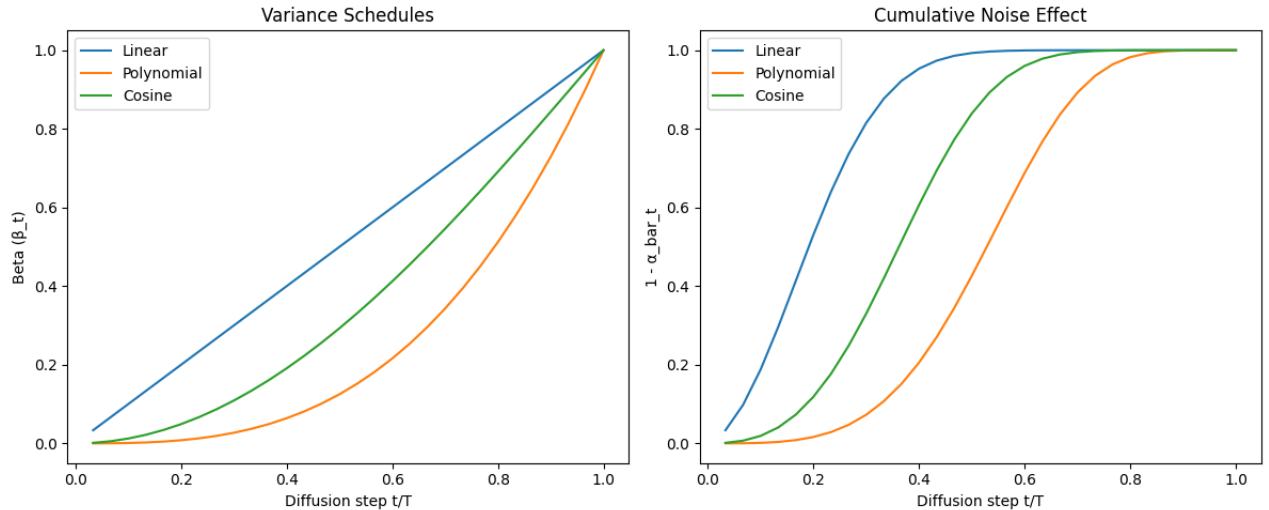
$$\tilde{\mu}_t(x_t, \epsilon_t) = \left(A + \frac{B}{\sqrt{\bar{\alpha}_t}} \right) x_t - B \sqrt{\frac{1 - \bar{\alpha}_t}{\bar{\alpha}_t}} \epsilon_t$$

Thus, the coefficients are:

$$A' = A + \frac{B}{\sqrt{\bar{\alpha}_t}} = \frac{\sqrt{\alpha_{t-1}}}{1 - \bar{\alpha}_t} + \frac{\sqrt{\beta_t(1 - \alpha_{t-1})}}{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_t)}$$

$$B' = -B \sqrt{\frac{1 - \bar{\alpha}_t}{\bar{\alpha}_t}} = -\frac{\sqrt{\beta_t(1 - \alpha_{t-1})}}{1 - \bar{\alpha}_t} \sqrt{\frac{1 - \bar{\alpha}_t}{\bar{\alpha}_t}}$$

Problem 1.3



Short Answer

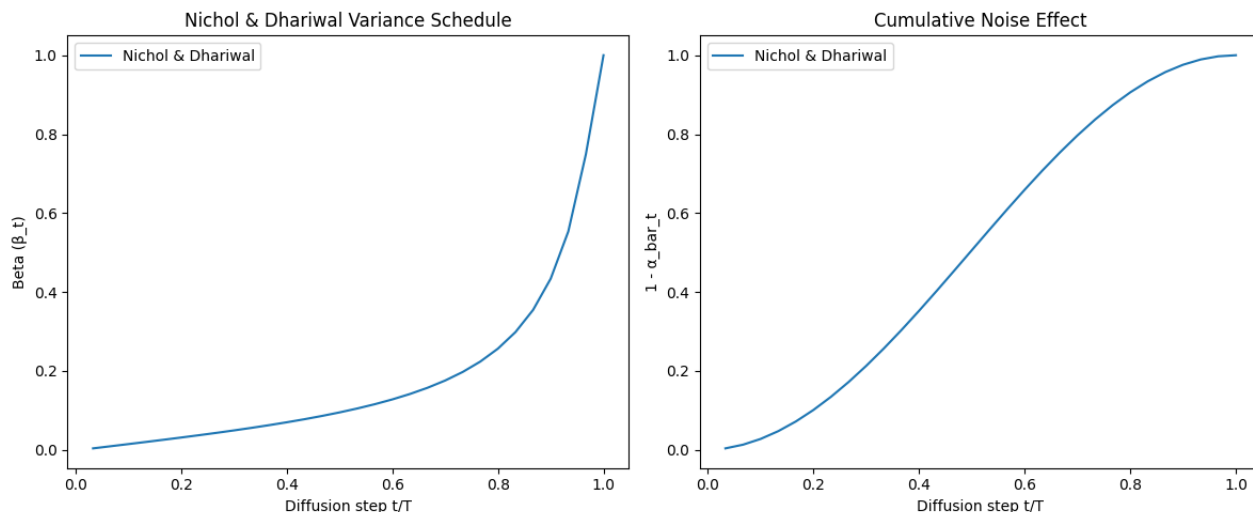
From the plots:

Linear Schedule: The noise increases gradually in a uniform manner, leading to a balanced trade-off between smoothness and destruction of structure. This is a common baseline.

Polynomial Schedule: Adds noise more slowly at early steps and increases sharply at later steps. This preserves more structure early in the diffusion process, potentially helping training stability.

Cosine Schedule: Adds noise very gradually at the beginning but quickly towards later steps. This allows for a longer denoising phase, which may help models generate more coherent samples.

Since $q(x_t|x_0)$ follows $\mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)\mathbf{I})$, schedules that make $\bar{\alpha}_t$ decrease more slowly early on will retain more signal for longer, potentially making training more effective. The cosine schedule is often favored in modern diffusion models due to its gradual noise introduction.



Short Answer

Compared to the previous schedules:

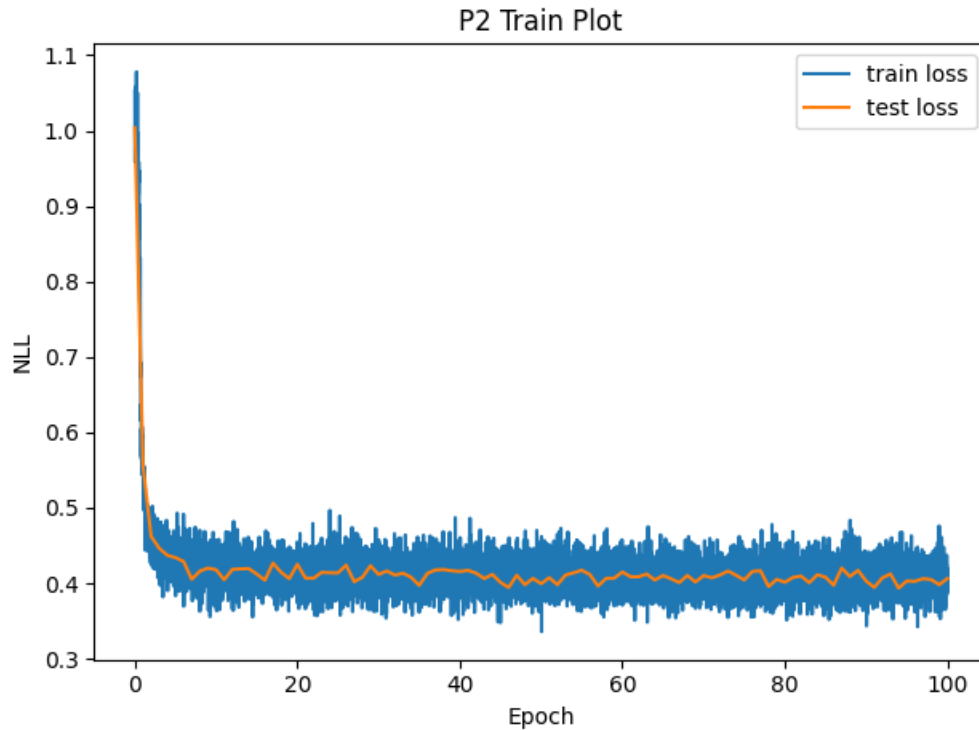
Smooth Transition: This schedule introduces noise more smoothly, preventing sudden jumps in noise levels.

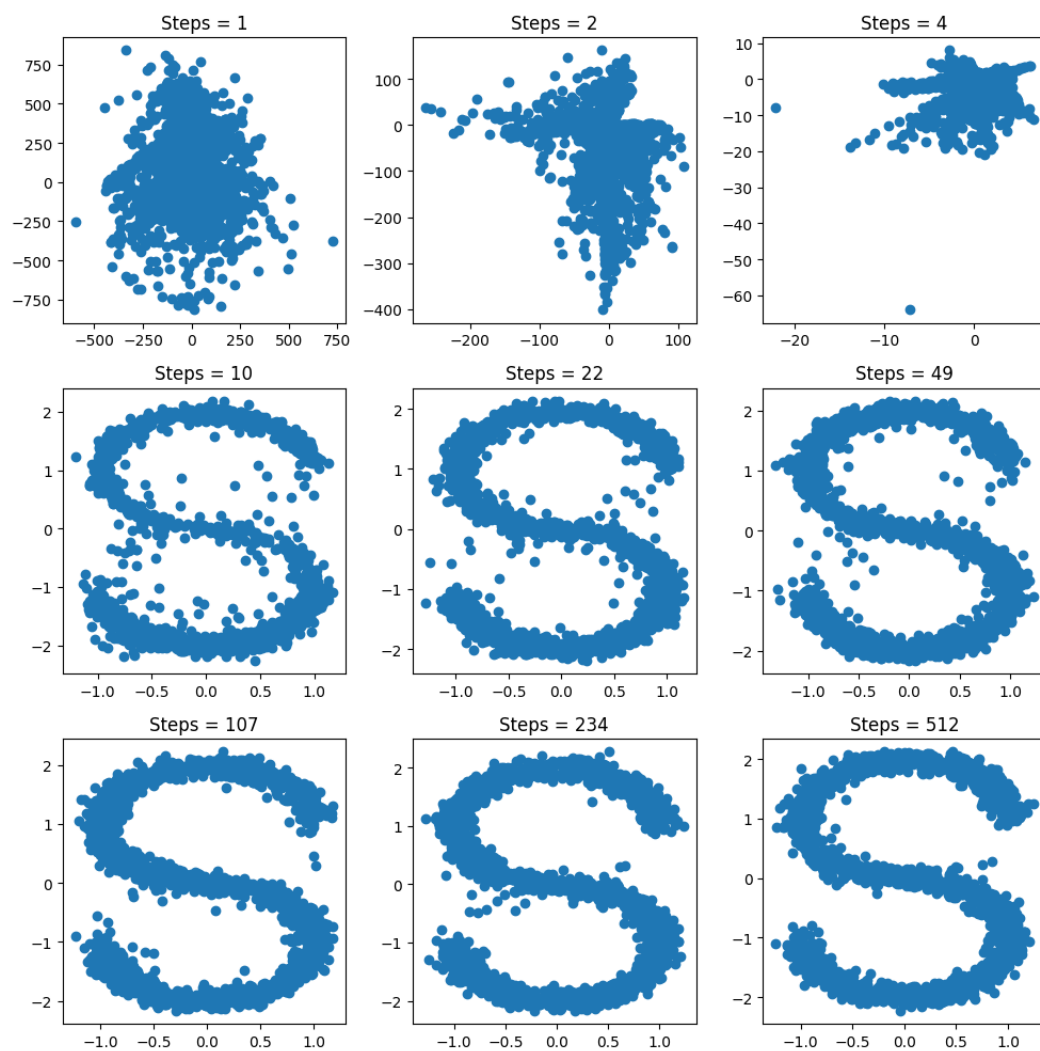
Better Preservation of Structure: The cosine-squared function ensures that $\bar{\alpha}_t$ decreases more gradually in early steps, preserving information for longer.

More Effective Training: This smooth noise transition helps prevent instability in training, leading to better final sample quality.

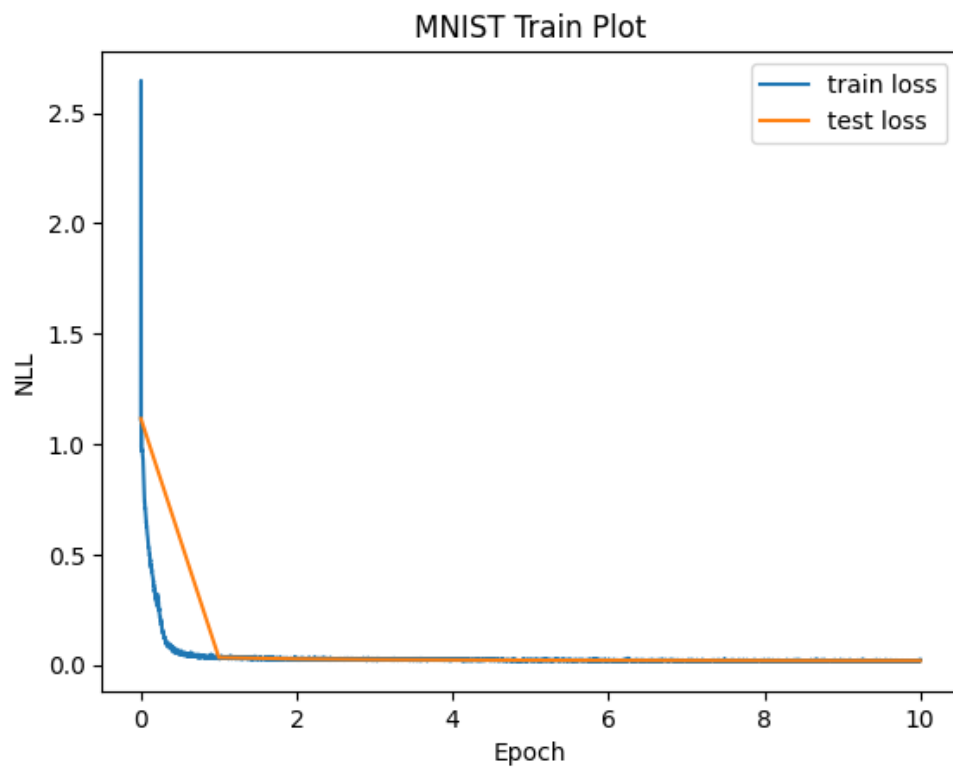
Nichol and Dhariwal’s schedule provides a well-balanced trade-off between early structure preservation and later noise accumulation, improving the efficiency of the diffusion process.

Part 2

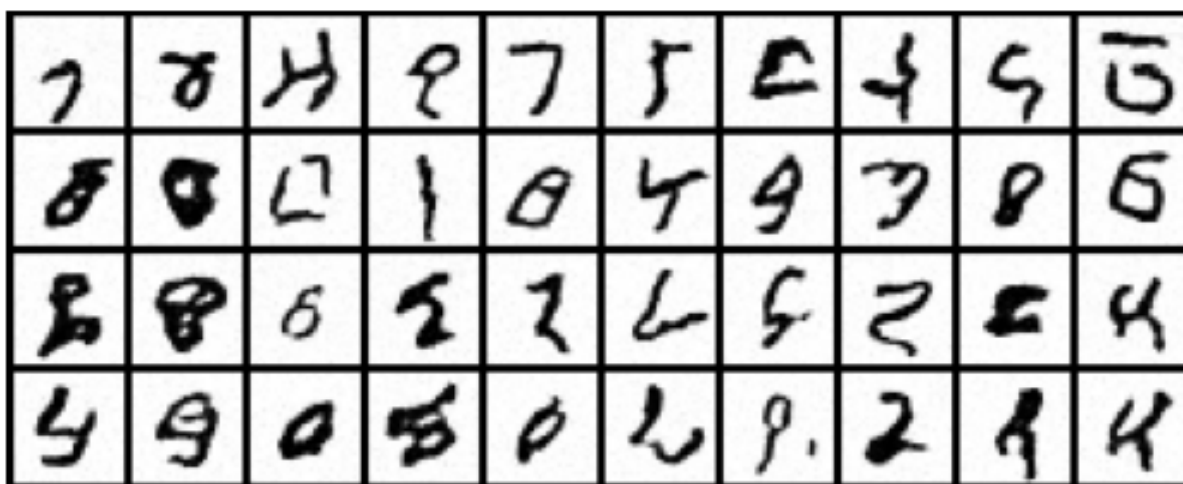




Part 3



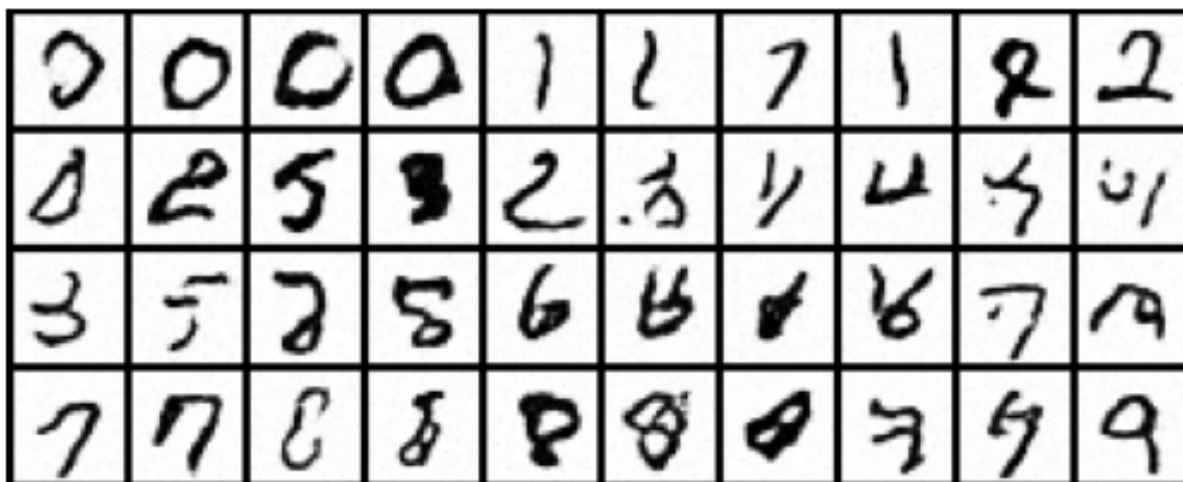
$w=0.0$



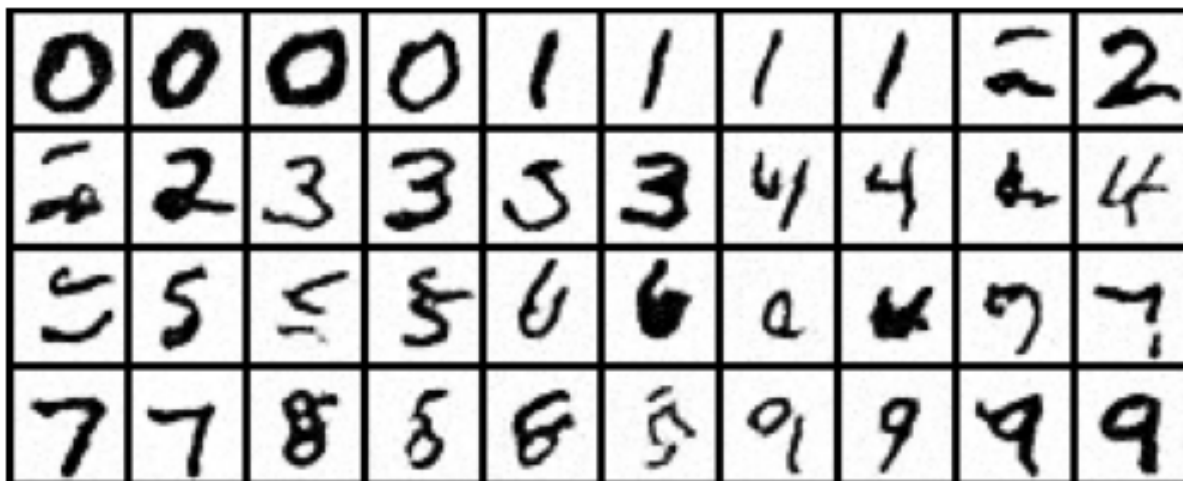
w=0.5



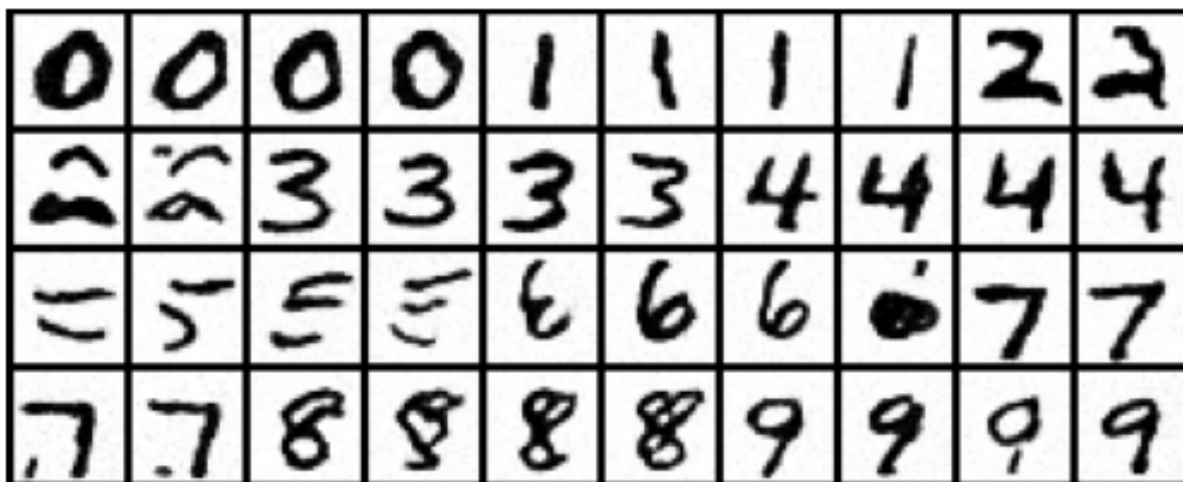
w=1.0



w=2.0



w=4.0



U-Net architecture

1. FourierEncoder: Encodes the scalar diffusion timestep t into a rich, high-dimensional vector embedding using sine and cosine functions. This helps the model capture the periodic structure of time and provides a smooth embedding space.

Input: t of shape $(bs, 1, 1, 1)$

Output: Embedding of shape (bs, dim)

How it works: Flattens t to shape $(bs, 1)$, multiplies it with learnable frequencies and applies sine and cosine transformations, concatenates these to form the final embedding, scaled by $\sqrt{2}$ for normalization.

2. ResidualLayer: Applies two residual convolutional blocks to transform feature maps, while injecting time and label (class) conditioning. Acts as the core computational block in encoder, decoder, and midcoder.

Input:

x : feature map (bs, c, h, w)

t_embed : time embedding (bs, t_embed_dim)

y_embed : class embedding (bs, y_embed_dim)

Output: Processed feature map (bs, c, h, w)

How it works: Applies a $conv \rightarrow norm \rightarrow activation$ block, adds time and class embeddings after transforming them to match the channel dimension, expanding to $(bs, c, 1, 1)$ and broadcasting across spatial dimensions, applies another $conv \rightarrow norm \rightarrow activation$, adds the original input (res) back as a residual connection.

3. Encoder: Downsamples feature maps while applying several ResidualLayers to extract higher-level representations and reduce spatial resolution.

Input:

x : feature map (bs, c_in, h, w)

t_embed : time embedding (bs, t_embed_dim)

y_embed : class embedding (bs, y_embed_dim)

Output: Downsampled feature map $(bs, c_out, h//2, w//2)$

How it works: Passes x through a list of ResidualLayers (all keep the same shape), then applies a convolution with $stride=2$ to reduce height and width by half.

4. Decoder: Upsamples feature maps, aiming to reconstruct the original spatial resolution while combining high-level and low-level information (via skip connections).

Input:

x : feature map (bs, c_in, h, w)

`t_embed`, `y_embed` as above

Output: Upsampled feature map (`bs`, `c_out`, `2h`, `2w`)

How it works: Uses bilinear upsampling followed by a convolution to increase spatial dimensions and match channels, applies multiple `ResidualLayers` for processing with time and class conditioning.

5. Midcoder: Processes the bottleneck (lowest-resolution) part of the U-Net to allow interaction between the most abstract representations. Acts as a bridge between encoder and decoder.

Input:

`x`: feature map (`bs`, `c`, `h`, `w`)

`t_embed`, `y_embed` as above

Output: Processed feature map of the same shape (`bs`, `c`, `h`, `w`)

How it works: Applies a series of `ResidualLayers`, just like encoder/decoder, but without changing resolution or channel size.

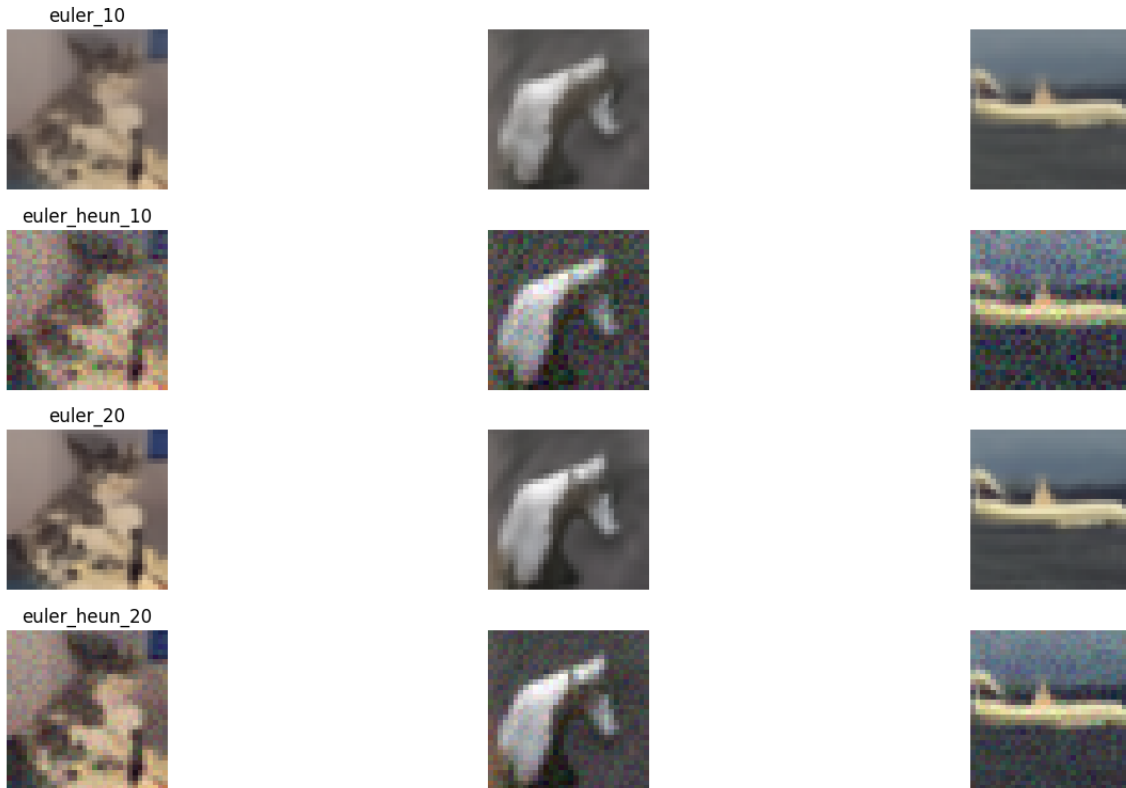
Tunable parameter w

Classifier-Free Guidance (CFG) introduces a tunable parameter w that interpolates between unconditional and fully conditional generation. As w increases, the model is more strongly guided toward the conditioning information (in this case, the digit label), which tends to improve alignment between the generated image and the desired class. For small values like $w = 0.0$, samples often lack recognizable digit structure, since the model ignores class information entirely. As w increases to values like 1.0 or 2.0, the samples become sharper, more coherent, and better reflect the intended digit label.

However, increasing w too much (e.g., $w = 4.0$) can sometimes lead to over-saturation or artifacts, where the model overcorrects toward class features and sacrifices sample diversity or realism. There's typically a sweet spot (often around $w = 1.0$ or 2.0) where both fidelity and class alignment are strong. This highlights CFG's value: it lets us balance creative freedom (lower w) with control and specificity (higher w) at inference time, without retraining the model.

Part 4

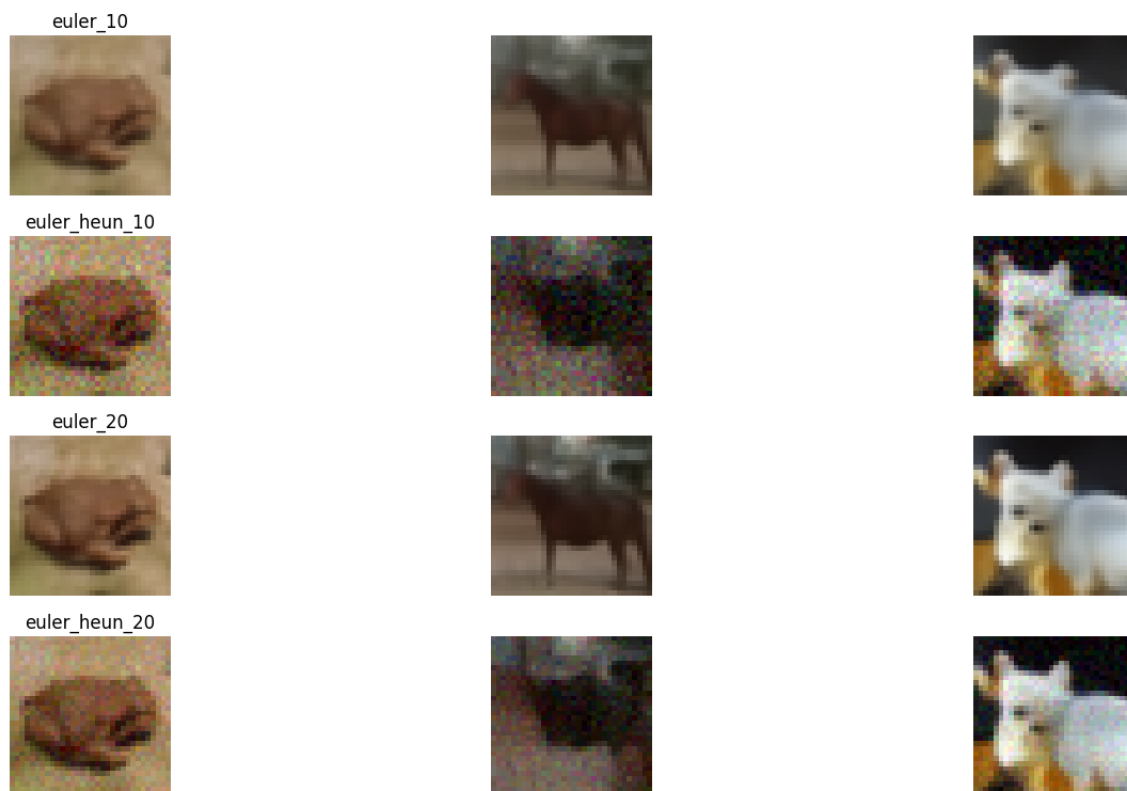
Comparison of Sampling Methods



Comparison of Sampling Methods



Comparison of Sampling Methods



Comparison of Sampling Methods



Short Answer

Note on the outputs above: I ran into OutOfMemory issues in trying to run steps=[10, 20, 50] for 16 images. I've instead included 6 images for 10 and 20 steps each, and 4 images for 30 steps, using 2 different seeds. I hope this is sufficient to capture the essence of this problem...

Using different samplers and varying the number of inference steps reveals a clear trade-off between sample quality and computational cost. With a small number of steps (e.g., 10), both Euler and Euler-Heun samplers may produce blurrier or noisier images, as the integration of the ODE is too coarse to accurately follow the score field. However, even at low step counts, the Euler-Heun sampler generally outperforms standard Euler by reducing artifacts and preserving more structure, thanks to its second-order correction.

As the number of steps increases, both samplers improve, with Euler-Heun continuing to show smoother gradients, sharper edges, and more consistent semantics. At high step counts, the performance gap narrows, but Euler-Heun can still yield marginally better fidelity or fewer anomalies. Overall, Heun correction provides a more accurate approximation of the reverse diffusion process, especially beneficial when inference budgets are tight and fewer steps are used.